基于 Starry 的机器人应用

在 SpacemiT K3 异构平台上复现 AI 调度并跑通 VLA / LLM

相恒硕　2023011125

2026-11-09

1. QEMU 复现 + StarryOS 启动
2. 真板启动 + 首次真板模型推理
3. TinyVLA 真板吞吐测试

选题与目标

题目：基于 Starry 的机器人应用
基准：k3x-HERA-RT + tgoskits（StarryOS）

K3 的架构

资源	规格	约束
X100 ×8	RVV，VLEN=256	通用计算主力
A100 ×8	RVV，VLEN=1024，含 IME/TCM	向量宽度不同，不能随意迁移
RT24	实时核	AI 推理不能干扰实时控制

阶段一：QEMU 复现

1. 启动

```
Welcome to Starry OS!
root@starry:/root #
```

- `/dev/k3_airunner` 存在；
- `k3_test` 建 channel，提交 `(a‖b)→d` DAG；
- 内核拓扑序执行 3 个 MatMul，token=42 回写；
- 结果 `[18 24 30; 48 66 84]` 与手工验算一致。

2. ggml 桥接

- 中期报告时的内容
- 从旧提交提取 `k3_ggml_bridge`，适配旧 runtime API；
- 修复 RHS 转置 stride（否则 worker 越界 panic）；
- 两个 case（行主序 / 转置）均 PASS。

阶段二：真板

- 平台：BananaPi BPI-SM10（SpacemiT K3）
- 存储/串口/文件注入链路脚本化

内核补丁①：打开用户态 RVV

问题：真板上用户态向量指令一律 SIGILL

原因：StarryOS 只设置 `sstatus.FS`（浮点），没有设置 `sstatus.VS`（向量）。

修复：在 axcpu 用户上下文初始化里把 `VS` 置为 `Initial`。

内核补丁②：保存向量上下文

新问题：只开 VS，多进程向量负载仍随机 SIGSEGV / PC 飞掉

根因：任务切换不保存 v0–v31；第二个任务用向量单元时覆盖前一个任务

修复

- `TaskContext` 增加 `VectorState`；
- 保存 v0–v31 + `vcsr/vtype/vl/vstart`；
- 复用浮点 Dirty/Clean/Initial 状态机；
- m1 逐寄存器搬运，VLEN 无关（256/1024 都能用）；
- 保存前清零 4 KB 镜像，防止高位泄漏

首次真板模型推理

链路：StarryOS → ramfs → 用户态 musl → llama.cpp → 输出

- 模型：TinyStories-3M Q4_K_M（8.2 MB）
- 二进制：riscv64 musl 静态 `llama-cli`（9.7 MB）
- 存储：ramfs 内存副本，规避 SD 并发缺陷

```
> Once upon a time
Mia heard, "Mom, what is how she would make a nap."
...
[ Prompt: 1.0 t/s | Generation: 0.0 t/s ]
```

机器人应用：TinyVLA 任务

- 16×16 网格，4–8 个墙，红/绿/蓝三个目标；
- 指令：17 词表、180 种组合，如 `navigate to the blue target`；
- 动作：上下左右四方向；
- 成功：走到指令颜色；走错颜色失败；64 步超时。

模型：MLP 640→128→64→4（约 90k 参数，权重 364 KB）

TinyVLA：训练与推理性能

训练：12,000 张地图, 24 epoch

- 没有在这个板子上，另外一个服务器离线训练

结果：动作准确率 99.85%，颜色 100%

- 8 进程 + 绑核情形下决策吞吐 1e5/s

总结

- 在 QEMU 复现了 **StarryOS + k3_airunner + ggml**
- 在真板跑通，并跑通 llama.cpp
- 用 **TinyVLA** 把“内核调度 → 机器人应用”跑起来